情報学群実験第 3 NetworkSimulator

学籍番号 1280391
細川 夏風

2026 年 4 月 19 日

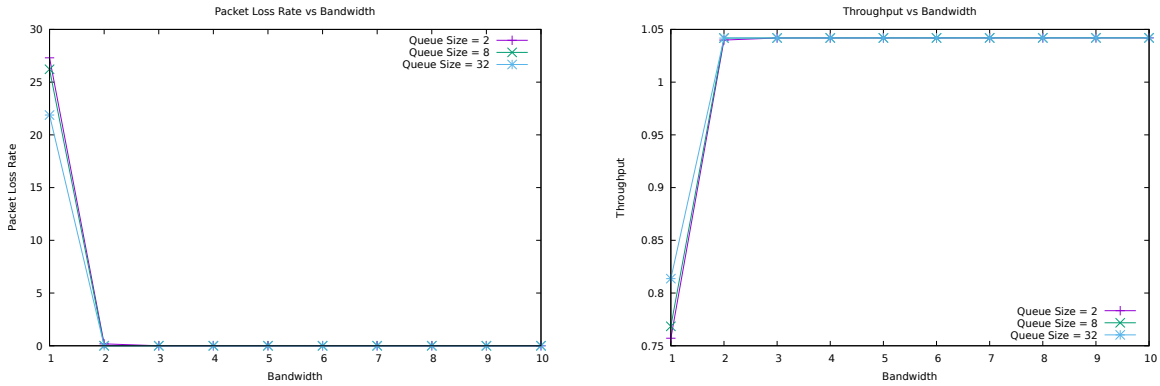
1　UDP 通信における帯域幅とキューサイズの影響

1.1　方法

UDP 通信の帯域幅，及びキューサイズを変更しその際のスループット，パケット損失率を観察する．また，この時の帯域幅は 1Mb から 1Mb ずつ上昇させ，キューサイズは 2KB から 32KB まで上昇させる．

1.2　結果

上記実験の結果を以下に示す．



(a) パケットロス (1-2)　　(b) スループット (1-2)

図 1: ネットワーク計測結果の比較

1.3 考察

上記の 2 図からわかることとして少量のキューサイズであったも上昇の影響は大きいということがわかる.

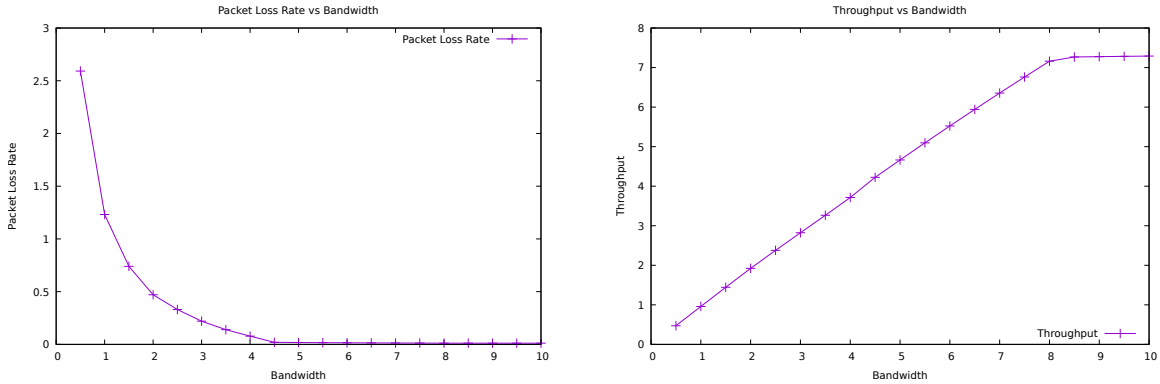
2 TCP 通信における帯域幅の影響と UDP 通信との比較

2.1 方法

TCP 通信の帯域幅を初期値 0.5Mb から最大値 10Mb まで 0.5Mb ずつ上昇させスループット，パケット損失率を観察する.

2.2 結果

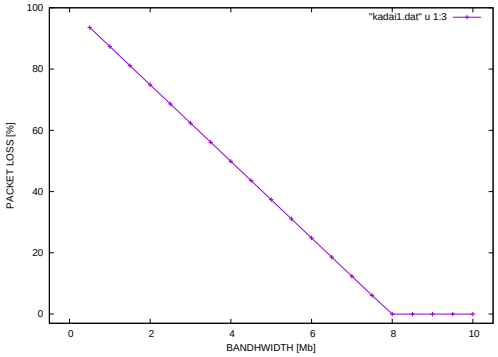
TCP 通信



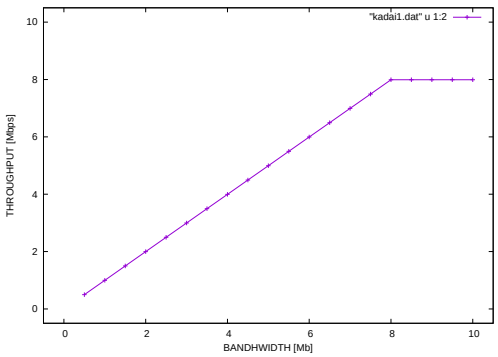
(a) パケットロス (1-3)　　(b) スループット (1-3)

図 2: TCP 通信の計測結果 (1-3)

UDP 通信



(a) パケットロス (UDP)



(b) スループット (UDP)

図 3: UDP 通信の計測結果

2.3 考察

以上の 2 図から，スループットには大幅な変化は無いことがわかる．これはスループットが単位時間あたりにおける通信量であるため，データ量や転送方法の影響を受けにくいことが理由として挙げられる．パケットロスについては TCP 通信の帯域幅 1 ～ 3 について大幅に減少していることとパケットロス自体が少ないことがわかる．これは TCP の持つ輻輳制御が理由であると考えられる．TCP には輻輳制御があり，パケットロスを減らす役割を持つ．そのため，通信に輻輳が検知されるとパケットの量を調整しパケットロスを減らしてくれるのだ．初期がパケットロスが TCP 通信では多いのはやはり，帯域幅が小さすぎるからであると考える．

3　追加データ

TCP 通信に含まれるデータからシーケンス番号 (seqno)，輻輳ウィンドウ (cwnd)，スロースタート閾値 (ssthresh) の値を抜き出して図に示す．

3.1　結果

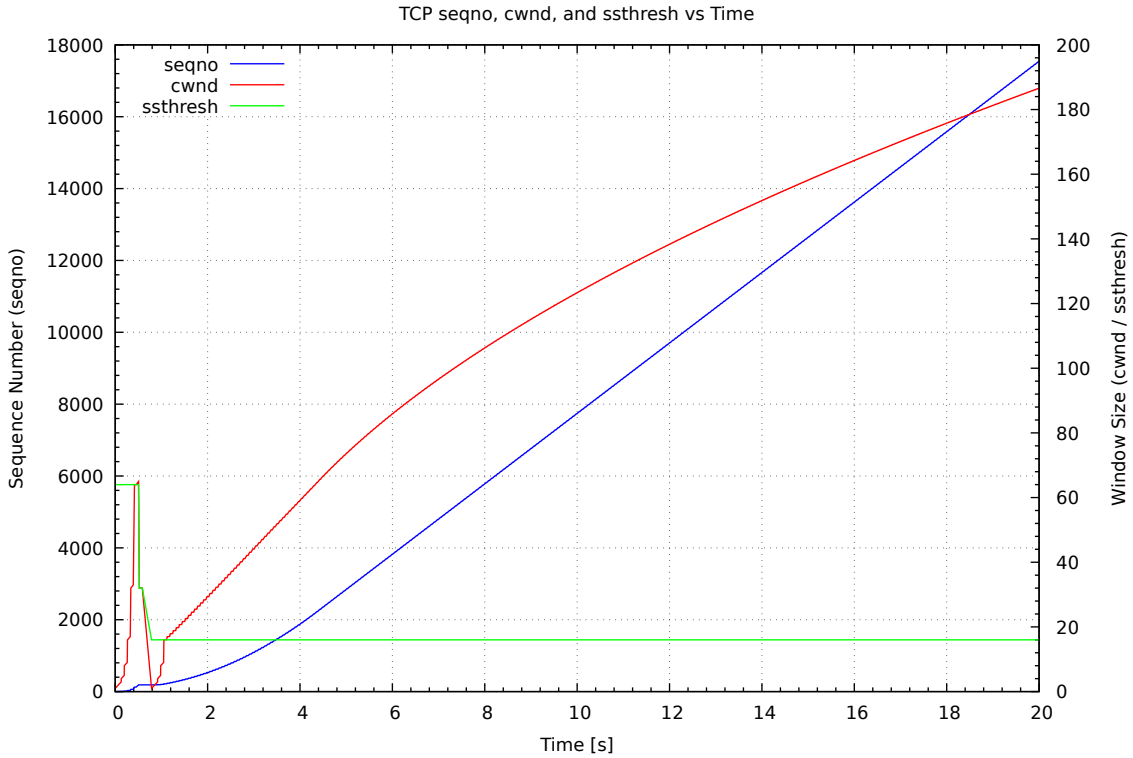


図 4: TCP 通信におけるシーケンス番号，輻輳ウィンドウ，スロースタート閾値の遷移

3.2　考察

図 4 から，シーケンス番号の値，輻輳ウィンドウのサイズが大きく減少している時間が存在している．これはおそらくタイムアウトが起こった状況であることがわかる．しかし，あらゆる手を尽くしてこのタイムアウト (不確定) のことについて考えて見たが確証のあるデータは得られなかった．ただ，同一のシーケンス番号の ACK が通常を遥かに超える回数各ノードに流れていることがわかった．これは本来ありえないことであり．これがタイムアウトの手がかかりになっていると考えられる．また，このデータから輻輳ウィンドウを超えると定数倍，正確には $\frac{1\mathrm{segment(Byte)}}{\mathrm{cwnd}} \times 1\mathrm{segment(Byte)}_{[1]}$ の値になっていることがわかる．

また，今回実験データの収集のために用いたプログラムは付録に掲載している．

参考文献

[1] マスタリング TCP/IP 入門篇 第 5 版，竹下隆史・村山公保・荒井透・苅田幸雄，株式会社オーム社，平成 26 年 5 月 20 日

A 付録：シェルスクリプト

A.1 パケット処理スクリプト

Listing 1: packet2.sh

```
#!/bin/sh

QUEUE=2

while [ $(echo "${QUEUE} <= 32" | bc) -eq 1 ]
do
  NUM=1
  while [ $(echo "${NUM} <= 10" | bc) -eq 1 ]
  do
    ns kadai1-2.tcl ${NUM}Mb ${QUEUE}

    sleep 2

    start=$(awk '$3 == 2 && $4 == 3 {print $2}' out.tr | head -n 1)
    end=$(awk '$3 == 2 && $4 == 3 {print $2}' out.tr | tail -n 1)
    time=$(echo "scale=6; ${end} - ${start}" | bc)

    re_packet=$(awk '$1 == "r" && $3 == 2 && $4 == 3' out.tr | wc -l)
    packet_size=$(awk '$1 == "r" && $3 == 2 && $4 == 3 {sum+=$6} END {
        print sum}' out.tr)

    packet_size=$(echo "scale=6; ${packet_size} * 8" | bc)
    throu_put=$(echo "scale=6; ${packet_size} / ${time} / 1000000" | bc)

```

```
    sent_packet=$(awk '$1 == "+" && $3 == 2 && $4 == 3' out.tr | wc -l)
    drop_packet=$(awk '$1 == "d" && $3 == 2 && $4 == 3' out.tr | wc -l)

    drop_rate=$(echo "scale=6; ${drop_packet} / ${sent_packet} * 100" |
        bc)

    echo "${NUM} ${QUEUE} ${throu_put} ${drop_rate}" >> kadai1-2-${QUEUE
        }.dat
    NUM=$(echo "${NUM} + 1" | bc)
  done
  QUEUE=$(echo "${QUEUE} * 4" | bc)
done
```

A.2 TCP パケット処理スクリプト

Listing 2: packet_TCP.sh

```
#!/bin/sh

NUM=0.5

while [ $(echo "${NUM} <= 10" | bc) -eq 1 ]
do

  ns kadai1-3.tcl ${NUM}Mb

  sleep 2

  start=$(awk '$3 == 2 && $4 == 3 {print $2}' out.tr | head -n 1)
  end=$(awk '$3 == 2 && $4 == 3 {print $2}' out.tr | tail -n 1)
  time=$(echo "scale=6; ${end} - ${start}" | bc)

  re_packet=$(awk '$1 == "r" && $3 == 2 && $4 == 3' out.tr | wc -l)
  packet_size=$(awk '$1 == "r" && $3 == 2 && $4 == 3 {sum+=$6} END {print
      sum}' out.tr)

  packet_size=$(echo "scale=6; ${packet_size} * 8" | bc)
```

```
  throu_put=$(echo "scale=6; ${packet_size} / ${time} / 1000000" | bc)

  sent_packet=$(awk '$1 == "+" && $3 == 2 && $4 == 3' out.tr | wc -l)
  drop_packet=$(awk '$1 == "d" && $3 == 2 && $4 == 3' out.tr | wc -l)

  drop_rate=$(echo "scale=6; ${drop_packet} / ${sent_packet} * 100" | bc)

  echo "${NUM} ${throu_put} ${drop_rate}"

  NUM=$(echo "${NUM} + 0.5" | bc)
done
```